

Projekt: egyszerű LZ77 tömörítő/dekompresszor

Rövid leírás

Ez a projekt egy egyszerű LZ77-szerű tömörítőt és visszafejtőt tartalmaz C nyelven. A cél az LZ77 alapötletének gyakorlása: előretekintő (lookahead) ablakból és egy cirkuláris előzmény (sliding window) puffert használva a bemenetet tokenekre bontjuk, amelyek távolságot, hosszúságot és egy következő nyers bájtot tartalmaznak. A kimenet bináris token-sorozat, amelyet a dekóder visszaállítja eredeti bájtsorozattá.

Mi működik már

- Beolvasás: Az `encode.c` beolvassa a teljes bemeneti fájlt memóriába (Ahead struktúrába) és kezeli annak méretét.
- Cirkuláris puffer: A `circular_buffer.c` / `circular_buffer.h` fájlokban található puffer és `add` függvény karban tartja az előzményi ablakot, illetve a `find_match` megkísérel hosszabb egyezéseket keresni a lookahead és a puffer között.
- Tokenezés: Az `encode.c` létrehoz `Token` struktúrákat, melyekben a `distance` (távolság), `length` (hossz) és `next` (következő nyers bájt) mezők tárolódnak. A kódrészlet kezeli a "nincs egyezés" esetet (`distance=0, length=0`).
- Kiírás: A `save` függvény binárisan kiír egy `Token`-t egy fájlba; az `encode` program tokenenként írja a kimeneti fájlt.
- Dekódolás: A `decoder.c` beolvassa a tokeneket a fájlból, és rekonstruálja a kimeneti bájtsorozatot, majd kiírja azt fájlba.

Mire kell figyelni / hiányosságok

- Stabilitás és biztonság: A kód jó célokra alkalmas, de nincs teljes hibakezelés minden esetben (például fájlok és memóriahibák részletes kezelése hiányos). A bemeneti fájlok méretének és a memórafoglalásnak nincsenek alapos korlátai minden részen.
- Bináris token formátum: A tokenek egyszerűen memcpy-vel íródnak ki, ami hordozhatósági problémákat okozhat platformok/endianness esetén. Jelenleg feltételezzük, hogy az író és olvasó ugyanazon a gépen fut.
- Limitációk az egyezéskeresésben: A `find_match` implementáció egyszerű és lehetnek benne logikai hibák vagy edge-case-ek (például rossz `d` érték inicializálás, vagy ha nincs egyezés). Érdemes tesztet futtatni különböző inputokra.

Fájlok és fontos függvények szerepe

- `encode.c`
 - Fő funkció: bemenet beolvasása, egyezések keresése, tokenek létrehozása és bináris kimenet írása.

- Fontosabb elemek:

- `Token` struct: a tömörített egység, mezői: `uint16_t distance; uint8_t length; unsigned char next; .`
- `read_file(const char *filename, Ahead *a) :` beolvassa a teljes fájlt az `Ahead` struktúrába (méret + puffer).
- `save(Token token, FILE *f, int token_num) :` binárisan kiír egy token szerkezetet a fájlba; debug-kiírásokat is tartalmaz bizonyos token indexeknél.
- A `main` ciklus: egyezés keresés `find_match` segítségével, a puffer frissítése `add` hívásokkal, tokenek mentése fájlba.

- `decoder.c`

- Fő funkció: bináris tokenfájl beolvasása és eredeti adat visszaállítása.

- Fontosabb elemek:

- `read_file` (a decoder verziója): beolvassa a tokeneket és rekonstruálja a kimeneti bájt-sorozatot egy előre lefoglalt pufferbe (a puffer méretét konzervatívan számítja).
- A dekódolás logikája: ha `length==0` és `distance==0`, akkor egy sima nyers bájtot vesz; különben a pufferből visszamásolja a megadott távolságból és hosszban az adatot, majd hozzáfűzi a `next` bájtot.

- `circular_buffer.h / circular_buffer.c`

- Fő funkció: előzményi ablak megvalósítása és egyszerű egyezéskeresés.

- Fontosabb elemek:

- `Buffer` struct: tárolja a puffer tartalmát (`data`), valamint `size`, `start`, `end` indexeket.
- `Ahead` struct: a lookahead adatait tartalmazza (méret és pointer).
- `add(char item, Buffer *b) :` hozzáad egy bájtot a cirkuláris pufferhez; ha a puffer megtelt, felülírja a legrégebbi elemet.
- `find_match(Buffer *b, Ahead *a) :` megpróbálja megtalálni a leghosszabb egyezést a `b` puffer és az `a` lookahead között. Visszaad egy `Match` struktúrát (távolság `d`, hossz `l`, és egy `c` mező, ami egy kapcsolódó karaktert tartalmaz).

Példa használat (parancssorban)

- Tömörítés:

- gcc használatával fordítsd le: például a VS Code feladat `C/C++: gcc.exe build active file` fordítja az aktív fájlt; de tipikusan mindkét programot le kell fordítani külön-külön:

```
gcc -g -o encode.exe encode.c circular_buffer.c
```

```
gcc -g -o decoder.exe decoder.c
```

- Futtatás:

```
./encode.exe example.txt example_out.lz77
```

```
./decoder.exe example_out.lz77 decoded.txt
```

Javasolt fejlesztési lépések (következő lépések)

- Tesztek készítése rövid és hosszú bemenetekre, bináris és szöveges fájlokra.
- Ellenőrizni és javítani a `find_match` logikát edge-case-ekre.
- Kezelní a platformfüggetlenséget (endianness, strukturális padding) – pl. explicit bájtok írása a token mezőkhöz hálózati (big-endian) vagy kis-endian formátumban.

Rövid összefoglaló

Ez a projekt egy működő, egyszerű LZ77-szerű encode/decode eszközt tartalmaz, alapvető read/write és pufferekési logikával. A jelenlegi implementáció oktatási célokra alkalmas, de még igényel hibakezelést, formátum-verziózást és alapos tesztelést production-szerű használathoz.